



INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

ASSIGNMENT

Assignment Title: WEP40 Wireless Packet Decryption and Aircrack Forensics

Student Name: Ahmad Zubairu Garba

Student ID: 2025/FWSD/11463

Course Title: Basic Networking Skills for Digital Forensics

Course Code: SBT-DF203

Instructor: Aminu Idris

September 25, 2026

1.EXECUTIVE OVERVIEW

This technical report details the offline forensic examination and cryptographic cryptanalysis of a historical IEEE 802.11 wireless packet capture (file.xz / file_working) encrypted via the deprecated Wired Equivalent Privacy (WEP40) protocol. The primary objective of this investigation was to evaluate the structural vulnerabilities of the RC4 stream cipher, perform offline key recovery without live frame injection, decrypt the underlying network payloads, and analyze the post-decryption Layer 2 through Layer 7 traffic artifacts.

The analysis was performed in an isolated, non-routing virtual laboratory running Kali Linux. Using the aircrack-ng framework, statistical cryptanalysis was applied against 15,477 captured Initialization Vectors (IVs), successfully extracting the 40-bit secret key in under one second. Subsequent decryption via airdecap-ng converted the raw 802.11 frame encapsulate into a standard IEEE 802.3/Ethernet PCAP structure (file_working-dec). Advanced network forensics using tshark and automated carving via foremost enabled complete endpoint identification, session reconstruction, and artifact extraction (including HTML documents, JavaScript files, and PNG images).

Evidence Inventory & Hash Ledger

To maintain strict forensic chain-of-custody and prevent evidence contamination, cryptographic SHA-256 digests were computed and logged immediately upon file acquisition, decompression, and decryption.

EVIDENCE STAGE	FILE SYSTEM PATH	FILE TYPE / DESCRIPTION	SHA-256 CRYPTOGRAPHIC HASH
ORIGINAL EVIDENCE	evidence/file.xz	XZ	dd54144caef34f228bfb6b87a9101ca7173969376f
		Compressed Capture Archive	7ed357bd068061d4f4b8d6
WORKING COPY (RAW)	working/file_working	Decompressed PCAP	c17a3f9b955e8df5befd476dbd55c67286d1e3eea9ab402d5359cac8874ebb2d

		(45,169 frames)	
DECRY	working/file_	Decrypte	167c91994c269777f9048227dcb89882caf3cf763
PTED	working-dec	d IEEE	977f2059604f0a6a340b804
CAPTU		802.11	
RE		Payload	

2. TECHNICAL LAB EXECUTION & EVIDENCE ANALYSIS

Phase 1: Environment Initialization & Forensic Preservation

A structured directory hierarchy was established under ~/SBT-DF203-Lab9 to segregate raw evidence, active working files, carved artifacts, and generation logs. The evidence archive was retrieved over HTTPS from the authorized repository, hashed, copied with preserved timestamps, and decompressed.

Terminal Command Executed:

```
mkdir -p ~/SBT-DF203-Lab9/{evidence,working,exported,reports,screenshots,scripts}
```

```
cd ~/SBT-DF203-Lab9
```

```
find . -maxdepth 1 -type d -print
```

The terminal output verifies the initialization of six isolated operational subdirectories.

The directory structure ensures strict segregation of read-only evidence from active analysis outputs.

```
(garba@kali)~$ mkdir -p ~/SBT-DF203-Lab9/{evidence,working,exported,reports,screenshots,scripts}
(garba@kali)~$ cd ~/SBT-DF203-Lab9
(garba@kali)~/SBT-DF203-Lab9$ pwd
/home/garba/SBT-DF203-Lab9
(garba@kali)~/SBT-DF203-Lab9$ find . -maxdepth 1 -type d -print
.
./exported
./screenshots
./scripts
./working
./evidence
./reports
(garba@kali)~/SBT-DF203-Lab9$
```

Evidence Acquisition & Cryptographic Hashing

Terminal Command Executed:

```
wget -O evidence/file.xz 'https://raw.githubusercontent.com/.../good-crypto/file.xz'
```

```
sha256sum evidence/file.xz | tee reports/file_xz_sha256.txt
```

```
cp --preserve=timestamps evidence/file.xz working/file_working.xz
```

```
unxz -k working/file_working.xz
```

```
file working/file_working
```

```
sha256sum working/file_working.xz working/file_working | tee reports/working_hashes.txt
```

The terminal verifies successful retrieval of 12.24 MB (12,837,660 bytes) of compressed data.

Decompression via `unxz` yields `working/file\_working`, identified by `file` as a TCPdump pcap

capture file version 2.4 (IEEE 802.11, capture length 65535). SHA-256 values confirm file preservation.

```
(garba@kali)~[~/SBT-DF203-Lab9]
$ wget -O evidence/file.xz 'https://raw.githubusercontent.com/ctfs/write-ups-2015/master/codegate-ctf-2015/programming/good-crypto/file.xz'
--2026-09-25 21:37:42-- https://raw.githubusercontent.com/ctfs/write-ups-2015/master/codegate-ctf-2015/programming/good-crypto/file.xz
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.111.133, 185.199.110.133, 185.199.109.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.111.133|:443 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 12837660 (12M) [application/octet-stream]
Saving to: 'evidence/file.xz'

evidence/file.xz      100%[=====>] 12.24M  785KB/s  in 13s

2026-09-25 21:37:57 (944 KB/s) - 'evidence/file.xz' saved [12837660/12837660]

(garba@kali)~[~/SBT-DF203-Lab9]
$ sha256sum evidence/file.xz | tee reports/file_xz_sha256.txt
dd54144caef34f228bfb4b87a9101ca7173969376f7ed357bd068061d4f4b8d6  evidence/file.xz

(garba@kali)~[~/SBT-DF203-Lab9]
$ cp --preserve=timestamps evidence/file.xz working/file_working.xz

(garba@kali)~[~/SBT-DF203-Lab9]
$ unxz -k working/file_working.xz

(garba@kali)~[~/SBT-DF203-Lab9]
$ file working/file_working
working/file_working: pcap capture file, microsecond ts (little-endian) - version 2.4 (802.11, capture length 65535)

(garba@kali)~[~/SBT-DF203-Lab9]
$ sha256sum working/file_working.xz working/file_working | tee reports/working_hashes.txt
dd54144caef34f228bfb4b87a9101ca7173969376f7ed357bd068061d4f4b8d6  working/file_working.xz
c17a3f9b955e84f5befd476dbd55c67286d1e3eea9ab402d5359cac0874ebb2d  working/file_working
```

CAPTURE INVENTORY & CRYPTOGRAPHIC VULNERABILITY ASSESSMENT

To evaluate the capture composition, capinfos was executed to generate metadata metrics, followed by tshark protocol hierarchy statistics.

PCAP Metadata & Protocol Breakdown

Terminal Command Executed:

```
CAP="working/file_working"
```

```
capinfos "$CAP" | tee reports/capinfos.txt
```

```
tshark -r "$CAP" -q -z io,phs | tee reports/protocol_hierarchy.txt
```

`capinfos` confirms 45,169 total packets spanning 272.97 seconds with an average data bit rate of

395 kbps. The protocol hierarchy confirms pure 802.11 encapsulation (`wlan`), with 15,713 data frames

carrying active network payloads.

```
(garba@kali) [~/SBT-DF203-Lab9]
$ CAP=working/file_working
$ capinfos "$CAP" | tee reports/capinfos.txt
tshark -r "$CAP" -q -z io,phs | tee reports/protocol_hierarchy.txt
File name: working/file_working
File type: Wireshark/tcpdump/... - pcap
File encapsulation: IEEE 802.11 Wireless LAN
File timestamp precision: microseconds (6)
Packet size limit: file hdr: 65535 bytes
Number of packets: 45 k
File size: 14 MB
Data size: 13 MB
Capture duration: 272.969795 seconds
Earliest packet time: 2015-03-06 08:51:54.364525
Latest packet time: 2015-03-06 08:56:27.334320
Data byte rate: 49 kBps
Data bit rate: 395 kbps
Average packet size: 299.13 bytes
Average packet rate: 165 packets/s
SHA256: c17a3f9b955e84f5befd476dbd55c67286d1e3eea9ab402d5359cac0874ebb2d
SHA1: cb929fd6e407c13639e3168b0f35f921788478d3
Strict time order: False
Number of interfaces in file: 1
Interface #0 info:
  Encapsulation = IEEE 802.11 Wireless LAN (20 - ieee-802-11)
  Capture length = 65535
  Time precision = microseconds (6)
  Time ticks per second = 1000000
  Number of stat entries = 0
  Number of packets = 45169

Protocol Hierarchy Statistics
Filter:
frame frames:45169 bytes:13511274
wlan frames:45169 bytes:13511274
data frames:15713 bytes:13104128
```

IEEE 802.11 Frame Sampling

Terminal Command Executed:

```
tshark -r "$CAP" -Y 'wlan' -T fields -e frame.number -e frame.time \
-e wlan.fc.type -e wlan.fc.subtype -e wlan.sa -e wlan.da -e wlan.bssid \
| head -n 100 | tee reports/wlan_frame_sample.tsv
```

Enumeration of the first 100 frames demonstrates standard 802.11 operations, displaying type

values corresponding to Management (Type 0), Control (Type 1), and Data (Type 2) frames.

```
(garba@kali) [~/SBT-DF203-Lab9]
$ tshark -r "$CAP" -Y 'wlan' -T fields -e frame.number -e frame.time -e wlan.fc.type -e wlan.fc.subtype -e wlan.sa -e wlan.da -e wlan.bssid \
| head -n 100 | tee reports/wlan_frame_sample.tsv
1 2015-03-06T08:51:54.364525000+0100 1 11
2 2015-03-06T08:51:54.364525000+0100 1 11
3 2015-03-06T08:51:54.364525000+0100 1 11
4 2015-03-06T08:51:54.364525000+0100 1 11
5 2015-03-06T08:51:54.364525000+0100 1 11
6 2015-03-06T08:51:54.364525000+0100 1 11
7 2015-03-06T08:51:54.364525000+0100 1 11
8 2015-03-06T08:51:54.364525000+0100 1 11
9 2015-03-06T08:51:54.364525000+0100 1 11
10 2015-03-06T08:51:54.364525000+0100 1 11
11 2015-03-06T08:51:54.364525000+0100 1 11
12 2015-03-06T08:51:54.364525000+0100 1 11
13 2015-03-06T08:51:54.364525000+0100 1 11
14 2015-03-06T08:51:54.364525000+0100 1 11
15 2015-03-06T08:51:54.364525000+0100 1 11
16 2015-03-06T08:51:54.364525000+0100 1 11
17 2015-03-06T08:51:54.364525000+0100 1 11
```

CRYPTOGRAPHIC MECHANICS OF WEP40 VULNERABILITIES

Filtering the dataset for protected frame headers (`wlan.fc.protected == 1`) confirmed that all transmission payloads were encrypted using WEP.

WEP relies on the RC4 stream cipher, combining a 24-bit cleartext Initialization Vector (IV) with a static 40-bit shared key to form a 64-bit keying state. The fundamental flaw lies in the short 24-bit length of the IV space, which permits only 2^{24} (16,777,216) unique values. On high-throughput networks, IV exhaustion occurs within minutes, forcing IV collisions. When an identical IV is reused with a static key, RC4 generates an identical keystream K :

$$C_1 = P_1 \oplus K$$

$$C_2 = P_2 \oplus K$$

An attacker intercepting two ciphertexts (C_1, C_2) encrypted with the same IV can eliminate the keystream completely:

$$C_1 \oplus C_2 = (P_1 \oplus K) \oplus (P_2 \oplus K) = P_1 \oplus P_2$$

This XOR sum exposes the plaintexts to frequency analysis and direct cryptographic attack.

WEP Protected Frame Extraction

Terminal Command Executed:

```
tshark -r "$CAP" -Y 'wlan.fc.protected==1' -T fields \
-e frame.number -e frame.time_epoch -e wlan.sa -e wlan.da -e wlan.bssid \
-e wlan.wep.iv -e wlan.wep.key | head -n 200 | tee reports/wep_protected_frames.tsv
```

Field extraction isolates 15,477 protected data frames. The output exposes the raw 24-bit Initialization Vectors (e.g., `0x256655`, `0xc2cbeb`) transmitted in cleartext ahead of the ciphertext payload.

```
(garba@kali) [~/SBT-DF203-Lab9]
$ tshark -r "$CAP" -Y 'wlan.fc.protected==1' -T fields \
-e frame.number -e frame.time_epoch -e wlan.sa -e wlan.da -e wlan.bssid -e wlan.wep.iv -e wlan.wep.key \
| head -n 200 | tee reports/wep_protected_frames.tsv
9432 1425628319.232982000 00:26:66:55:97:d4 ff:ff:ff:ff:ff:ff 00:26:66:55:97:d6 0x67ffec 0
10130 1425628320.138252000 f0:f6:1c:68:96:7c 00:26:66:55:97:d6 00:26:66:55:97:d6 0xc2cbeb 0
10155 1425628320.176656000 f0:f6:1c:68:96:7c 04:8d:38:48:e5:b4 00:26:66:55:97:d6 0xc3cbeb 0
10157 1425628320.177168000 f0:f6:1c:68:96:7c 04:8d:38:48:a8:b5 00:26:66:55:97:d6 0xc4cbeb 0
10159 1425628320.177168000 f0:f6:1c:68:96:7c 08:10:77:92:7c:2f 00:26:66:55:97:d6 0xc5cbeb 0
10161 1425628320.177168000 f0:f6:1c:68:96:7c 00:26:66:55:97:d4 00:26:66:55:97:d6 0xc6cbeb 0
10163 1425628320.177168000 f0:f6:1c:68:96:7c c8:3a:35:58:c4:ba 00:26:66:55:97:d6 0xc7cbeb 0
10165 1425628320.177167000 f0:f6:1c:68:96:7c ff:ff:ff:ff:ff:ff 00:26:66:55:97:d6 0xc8cbeb 0
10167 1425628320.177686000 00:26:66:55:97:d4 f0:f6:1c:68:96:7c 00:26:66:55:97:d6 0x68ffec 0
10170 1425628320.180760000 f0:f6:1c:68:96:7c ff:ff:ff:ff:ff:ff 00:26:66:55:97:d6 0x69ffec 0
10171 1425628320.182288000 f0:f6:1c:68:96:7c 33:33:00:00:00:02 00:26:66:55:97:d6 0xc9cbeb 0
10172 1425628320.187417000 00:26:66:55:97:d4 ff:ff:ff:ff:ff:ff 00:26:66:55:97:d6 0x6affec 0
10173 1425628320.187408000 f0:f6:1c:68:96:7c 33:33:00:00:00:02 00:26:66:55:97:d6 0xc9cbeb 0
10175 1425628320.187919000 f0:f6:1c:68:96:7c ff:ff:ff:ff:ff:ff 00:26:66:55:97:d6 0xcacbeb 0
10177 1425628320.188952000 f0:f6:1c:68:96:7c 33:33:00:00:00:02 00:26:66:55:97:d6 0x6bffec 0
10178 1425628320.189976000 f0:f6:1c:68:96:7c ff:ff:ff:ff:ff:ff 00:26:66:55:97:d6 0x6cffec 0
10183 1425628320.192016000 f0:f6:1c:68:96:7c ff:ff:ff:ff:ff:ff 00:26:66:55:97:d6 0xcbcbeb 0
10185 1425628320.193048000 f0:f6:1c:68:96:7c ff:ff:ff:ff:ff:ff 00:26:66:55:97:d6 0x6dffec 0
10224 1425628320.259088000 f0:f6:1c:68:96:7c ff:ff:ff:ff:ff:ff 00:26:66:55:97:d6 0xcccbeb 0
10228 1425628320.262680000 f0:f6:1c:68:96:7c ff:ff:ff:ff:ff:ff 00:26:66:55:97:d6 0x6effec 0
10229 1425628320.262678000 00:26:66:55:97:d4 f0:f6:1c:68:96:7c 00:26:66:55:97:d6 0x6fffec 0
10233 1425628320.263184000 f0:f6:1c:68:96:7c 00:26:66:55:97:d4 00:26:66:55:97:d6 0xcdcbeb 0
10261 1425628320.419350000 f0:f6:1c:68:96:7c 00:26:66:55:97:d4 00:26:66:55:97:d6 0xc6cbeb 0
10266 1425628320.420374000 f0:f6:1c:68:96:7c 01:00:5e:00:00:02 00:26:66:55:97:d6 0xcfcbeb 0
10268 1425628320.420374000 f0:f6:1c:68:96:7c 01:00:5e:00:00:fb 00:26:66:55:97:d6 0xd0cbeb 0
10270 1425628320.420886000 f0:f6:1c:68:96:7c 01:00:5e:00:00:02 00:26:66:55:97:d6 0x70ffec 0
10272 1425628320.422221000 f0:f6:1c:68:96:7c 01:00:5e:00:00:fb 00:26:66:55:97:d6 0x71ffec 0
```

Statistical Analysis of Repeated IVs

Terminal Command Executed:

```
tshark -r "$CAP" -Y 'wlan.fc.protected==1' -T fields -e wlan.wep.iv \
| sort | uniq -c | sort -nr | head -n 30 | tee reports/repeated_iv_summary.txt
```

Quantitative frequency distribution proves severe IV repetition within the 4.5-minute sample window.

Specific IV values such as `0x4208cb` and `0x28d3eb` recur up to 7 times each, confirming conditions for statistical key recovery.

A terminal window with a dark background and a Kali Linux logo watermark. The prompt is (garba@kali) [~/SBT-DF203-Lab9]. The command executed is tshark -r "\$CAP" -Y 'wlan.fc.protected==1' -T fields -e wlan.wep.iv \ | sort | uniq -c | sort -nr | head -n 30 | tee reports/repeated_iv_summary.txt. The output lists 30 IV values with their frequencies, sorted in descending order. The top two values are 0x4208cb and 0x28d3eb, both appearing 7 times. Other values include 0x95dfcb (6 times), 0x9415ed (5 times), 0x86e5eb (5 times), 0x7ae5eb (5 times), 0x28e7eb (5 times), 0x19e5eb (5 times), 0xd5d7eb (4 times), 0xd3e8eb (4 times), 0xc915ed (4 times), 0xc7d8eb (4 times), 0xade4eb (4 times), 0x73e5eb (4 times), 0x65d8eb (4 times), 0x4ad5eb (4 times), and 0x31e7eb (4 times).

```
(garba@kali) [~/SBT-DF203-Lab9]
$ tshark -r "$CAP" -Y 'wlan.fc.protected==1' -T fields -e wlan.wep.iv \
| sort | uniq -c | sort -nr | head -n 30 | tee reports/repeated_iv_summary.txt
 7 0x4208cb
 7 0x28d3eb
 6 0x95dfcb
 5 0x9415ed
 5 0x86e5eb
 5 0x7ae5eb
 5 0x28e7eb
 5 0x19e5eb
 4 0xd5d7eb
 4 0xd3e8eb
 4 0xc915ed
 4 0xc7d8eb
 4 0xade4eb
 4 0x73e5eb
 4 0x65d8eb
 4 0x4ad5eb
 4 0x31e7eb
```

AUTOMATED CRYPTANALYSIS & OFFLINE DECRYPTION

Because the dataset contained 15,477 protected IVs, aircrack-ng was executed against file_working. Aircrack utilizes Fluhrer, Mantin, and Shamir (FMS) attacks along with PTW cryptographic techniques to correlate known bytes of key state across reused IVs.

Aircrack-ng Key Recovery Execution

Terminal Command Executed:

```
aircrack-ng "$CAP" | tee reports/aircrack_output.txt
```

`aircrack-ng` targeted ESSID `cgnetwork` at BSSID `00:26:66:55:97:D6`. Analyzing 15,477 IVs across

83 tested key states yielded instantaneous key recovery: `KEY FOUND! [ A4:3D:F6:F3:74 ]` with 100% depth accuracy.

```
(garba@kali)-[~/SBT-DF203-Lab9]
$ aircrack-ng "$CAP" | tee reports/aircrack_output.txt
Reading packets, please wait ...
Opening working/file_working
Read 45169 packets.

# BSSID      ESSID      Encryption
1 00:26:66:55:97:D6  cgnetwork  WEP (15477 IVs)

Choosing first network as target.

Reading packets, please wait ...
Opening working/file_working
Read 45169 packets.

1 potential targets

Attack will be restarted every 5000 captured ivs.

Aircrack-ng 1.7

[00:00:00] Tested 83 keys (got 15477 IVs)

KB  depth  byte(vote)
0  0/ 1  A4(22784) 62(20992) A8(19968) B6(19968) 42(19456) 6E(19456) 91(19200) B7(19200) 26(18944) 68(18944) 9E(18944) 0E(18688)
1  0/ 1  3D(23040) 51(20736) 07(20480) 62(19968) 78(19968) 1F(19712) B0(19712) BD(19456) 80(19200) 85(19200) 9D(19200) 13(18944)
2  0/ 1  F6(23808) E4(20992) D0(20736) 68(20224) 95(19712) 38(19456) 0C(19200) 45(18944) 4F(18944) A8(18944) F7(18944) 60(18688)
3  1/ 10  F3(20480) C5(19968) D0(19968) 3E(19712) 43(19456) 52(19456) B2(19456) 09(19456) 20(19456) 8F(19200) A0(19200) B0(19200)
4  6/ 9  01(19712) 20(19456) 3E(19456) 52(19456) 5C(19456) 90(19456) F9(19456) 45(18944) 85(18944) 95(18944) D3(18944) 3A(18688)

KEY FOUND! [ A4:3D:F6:F3:74 ]
Decrypted correctly: 100%
```

Frame Payload Decryption via airdecap-ng

Using the recovered hexadecimal string (A43DF6F374), airdecap-ng processed the capture
Airdecap-ng Decryption & Output Hashing]

Terminal Command Executed:

```
cd working
```

```
airdecap-ng -w A43DF6F374 file_working | tee ../reports/airdecap_output.txt
```

```
cd ..
```

```
find working -maxdepth 1 -type f | tee reports/decrypted_files_inventory.txt
```

```
sha256sum working/* | tee reports/all_working_file_hashes.txt
```

`airdecap-ng` successfully decrypted all 15,477 WEP data packets without corruption, generating

`working/file\_working-dec`. SHA-256 hashing logged in
`reports/all\_working\_file\_hashes.txt` confirms payload integrity.

```
(garba@kali)-[~/SBT-DF203-Lab9]
$ cd working
airdecap-ng -w A43DF6F374 file_working | tee ../reports/airdecap_output.txt
cd ..
Total number of stations seen      10
Total number of packets read      45169
Total number of WEP data packets   15477
Total number of WPA data packets   0
Number of plaintext data packets   0
Number of decrypted WEP packets    15477
Number of corrupted WEP packets    0
Number of decrypted WPA packets    0
Number of bad TKIP (WPA) packets   0
Number of bad CCMP (WPA) packets   0

(garba@kali)-[~/SBT-DF203-Lab9]
$ find working -maxdepth 1 -type f | tee reports/decrypted_files_inventory.t
working/file_working
working/file_working-dec
working/file_working.xz

(garba@kali)-[~/SBT-DF203-Lab9]
$ sha256sum working/* | tee reports/all_working_file_hashes.txt
c17a3f9b955e84f5befd476dbd55c67286d1e3eea9ab402d5359cac0874ebb2d  working/file_working
167c91994c269777f9048227deb89882caf3cf3c763977f2059604f9a6a40b04  working/file_working-dec
dd54144caef34f228bf4b87a9101ca7173969376f7ed357bd068061d4f4b8d6  working/file_working.xz
```

NETWORK LAYER ENDPOINT & PROTOCOL FORENSICS

With the IEEE 802.11 payload decrypted, higher-layer protocols (ARP, IP, DNS, TCP, HTTP, TLS) became fully accessible for dissection.

Layer 2 & Layer 3 Endpoint Enumeration

File Name: Screenshot (468).jpg

Terminal Command Executed:

```
DEC="working/file_working-dec"
```

```
tshark -r "$DEC" -q -z endpoints,eth | tee reports/ethernet_endpoints.txt
```

```
tshark -r "$DEC" -q -z endpoints,ip | tee reports/ip_endpoints.txt
```

Ethernet statistics reveal that MAC address `68:96:7b:c8:2a:7a` (Apple, Inc.) transmitted 15,407 frames (12 MB). IPv4 statistics link this physical adapter directly to internal IP host `192.168.0.15`.

```
(garba@kali) [~/SBT-DF203-Lab9]
$ DEC=$(find working -maxdepth 1 -type f -name "*.dec" | head -n 1)
echo "Decrypted capture: $DEC" | tee reports/decrypted_capture_path.txt
Decrypted capture: working/file_working-dec

(garba@kali) [~/SBT-DF203-Lab9]
$ tshark -r "$DEC" -q -z endpoints,eth | tee reports/ethernet_endpoints.txt

Ethernet Endpoints
Filter: <No Filter>

```

	Packets	Bytes	Tx Packets	Tx Bytes	Rx Packets	Rx Bytes
Apple_68:96:7c	15407	12 MB	7076	616 kB	8331	11 MB
EFMNetworks_55:97:d6	15020	12 MB	8178	11 MB	6842	593 kB
ASUSTekCOMPU_2a:c2:7a	398	281 kB	223	279 kB	135	11 kB
IPv6mcast_fb	42	6,648 bytes	0	0 bytes	42	6,648 bytes
IPv6mcast_fb	36	7,022 bytes	0	0 bytes	36	7,022 bytes
Broadcast	31	3,386 bytes	0	0 bytes	31	3,386 bytes
IPv6mcast_7f:ff:fa	31	11 kB	0	0 bytes	31	11 kB
IPv6mcast_16	10	920 bytes	0	0 bytes	10	920 bytes
IPv6mcast_02	9	630 bytes	0	0 bytes	9	630 bytes
IPv6mcast_16	3	180 bytes	0	0 bytes	3	180 bytes
IPv6mcast_02	2	92 bytes	0	0 bytes	2	92 bytes
NetisTechnol_48:e5:b4	1	42 bytes	0	0 bytes	1	42 bytes
NetisTechnol_48:a8:b5	1	42 bytes	0	0 bytes	1	42 bytes
08:10:77:92:7c:2f	1	42 bytes	0	0 bytes	1	42 bytes
TendaTechnol_58:c4:ba	1	42 bytes	0	0 bytes	1	42 bytes
IPv6mcast_ff:2a:c2:7a	1	78 bytes	0	0 bytes	1	78 bytes

```
(garba@kali) [~/SBT-DF203-Lab9]
$ tshark -r "$DEC" -q -z endpoints,ip | tee reports/ip_endpoints.txt

IPv4 Endpoints
Filter: <No Filter>

```

	Packets	Bytes	Tx Packets	Tx Bytes	Rx Packets	Rx Bytes
192.168.0.15	15389	12 MB	6995	608 kB	8314	11 MB
198.90.20.111	8057	6,916 kB	4446	6,637 kB	3611	279 kB
199.27.79.193	5694	4,825 kB	3131	4,603 kB	2563	221 kB
192.168.0.9	329	277 kB	197	265 kB	132	11 kB
173.194.127.141	153	109 kB	80	102 kB	73	7,364 bytes
173.194.127.55	143	89 kB	71	82 kB	72	6,990 bytes
8.8.8.8	81	10 kB	40	6,960 bytes	41	3,291 bytes
199.16.158.186	66	15 kB	29	10 kB	37	5,214 bytes
173.194.127.227	66	24 kB	26	15 kB	40	8,455 bytes
173.194.127.229	55	22 kB	23	13 kB	32	9,721 bytes
31.13.70.1	51	19 kB	26	15 kB	25	3,151 bytes
173.241.248.220	44	6,848 bytes	20	3,122 bytes	24	3,726 bytes
198.41.208.140	44	28 kB	21	25 kB	23	2,718 bytes

TCP Conversation Analysis

Terminal Command Executed:

```
tshark -r "$DEC" -q -z conv,tcp | tee reports/tcp_conversations.txt
```

Session analysis details active socket pairs. Client `192.168.0.15` established major data flows

with web servers `198.90.20.111:80` (2,235 frames, 6.91 MB) and `199.27.79.193:80` (2,168 frames, 4.825 MB).

```
(garba@kali)-[~/SBT-DF203-Lab9]
$ tshark -r "$DEC" -q -z conv,tcp | tee reports/tcp_conversations.txt
```

TCP Conversations									
Filter:<No Filter>									
		←		→		Total		Relative	Duration
		Frames	Bytes	Frames	Bytes	Frames	Bytes	Start	
192.168.0.15:51007	↔ 198.90.20.111:80	1257	1,885 kB	978	73 kB	2235	1,958 kB	113.588792000	45.4035
192.168.0.15:50959	↔ 199.27.79.193:80	1209	1,816 kB	959	71 kB	2168	1,887 kB	21.074236000	34.8633
192.168.0.15:51008	↔ 198.90.20.111:80	1159	1,739 kB	947	70 kB	2106	1,810 kB	113.588792000	46.0179
192.168.0.15:51001	↔ 199.27.79.193:80	1145	1,725 kB	927	66 kB	2072	1,791 kB	63.066042000	26.6088
192.168.0.15:51009	↔ 198.90.20.111:80	687	1,028 kB	572	44 kB	1259	1,072 kB	113.605175000	28.7983
192.168.0.15:51014	↔ 198.90.20.111:80	664	991 kB	566	44 kB	1230	1,036 kB	114.020474000	25.0538
192.168.0.15:51006	↔ 198.90.20.111:80	654	970 kB	520	40 kB	1174	1,011 kB	113.578039000	25.4942
192.168.0.15:51000	↔ 199.27.79.193:80	375	561 kB	323	25 kB	698	586 kB	58.731642000	4.4552
192.168.0.15:50999	↔ 199.27.79.193:80	206	303 kB	180	15 kB	386	319 kB	56.792636000	1.9390
192.168.0.15:51036	↔ 192.168.0.9:5000	98	144 kB	70	5,705 bytes	168	150 kB	243.325633000	0.1777
192.168.0.15:50958	↔ 199.27.79.193:80	73	93 kB	67	12 kB	140	105 kB	20.145401000	14.9846
192.168.0.15:51035	↔ 192.168.0.9:5000	80	116 kB	56	4,756 bytes	136	121 kB	243.281602000	0.1398
192.168.0.15:50983	↔ 173.194.127.141:80	71	99 kB	59	4,392 bytes	130	104 kB	27.831547000	31.4188
192.168.0.15:50997	↔ 199.27.79.193:80	62	87 kB	37	3,826 bytes	99	90 kB	56.126972000	11.0492
192.168.0.15:51034	↔ 173.194.127.229:443	23	13 kB	32	9,731 bytes	55	22 kB	234.384001000	6.7568
192.168.0.15:51005	↔ 198.90.20.111:80	25	22 kB	28	4,757 bytes	53	27 kB	113.019956000	2.8111
192.168.0.15:50960	↔ 31.13.70.1:443	26	15 kB	25	3,151 bytes	51	19 kB	21.329724000	32.8418
192.168.0.15:51028	↔ 173.194.127.227:443	18	11 kB	29	7,603 bytes	47	18 kB	172.217100000	37.9452
192.168.0.15:51003	↔ 198.41.208.140:80	21	25 kB	23	2,718 bytes	44	28 kB	102.780857000	32.8878
192.168.0.15:51018	↔ 173.194.127.55:80	20	24 kB	16	1,494 bytes	36	26 kB	115.102391000	30.4286
192.168.0.15:50973	↔ 199.16.158.186:443	15	5,156 bytes	19	2,640 bytes	34	7,796 bytes	25.665085000	31.5233
192.168.0.15:51002	↔ 104.74.158.12:80	18	20 kB	15	1,236 bytes	33	21 kB	63.175099000	30.4746
192.168.0.15:50974	↔ 199.16.158.186:443	14	5,082 bytes	18	2,574 bytes	32	7,656 bytes	25.665085000	31.5223
192.168.0.15:51016	↔ 173.194.127.55:80	13	15 kB	19	1,749 bytes	32	16 kB	115.101879000	30.4439
192.168.0.15:51020	↔ 74.125.203.156:80	16	17 kB	16	2,534 bytes	32	20 kB	115.146935000	31.5115
192.168.0.15:50956	↔ 198.41.209.137:80	13	14 kB	15	1,556 bytes	28	16 kB	13.669693000	31.2704
192.168.0.15:51032	↔ 173.194.127.231:443	11	5,123 bytes	16	1,409 bytes	27	6,532 bytes	210.917057000	0.2191
192.168.0.15:50954	↔ 17.172.239.31:443	12	3,973 bytes	13	2,531 bytes	25	6,504 bytes	1.353792000	2.6631
192.168.0.15:51015	↔ 173.194.127.55:80	13	14 kB	12	1,230 bytes	25	15 kB	115.100855000	30.4291

Frame-Level IP-to-MAC Mapping Sample

Terminal Command Executed:

```
tshark -r "$DEC" -Y 'ip' -T fields -e frame.number -e wlan.sa -e wlan.da \
-e ip.src -e ip.dst -e _ws.col.Protocol | head -n 200 | tee
reports/ip_mac_mapping_sample.tsv
```

Field-extracted sequential logs document client `192.168.0.15` initiating DHCP requests, querying Google

DNS (`8.8.8.8`), generating IGMPv2 multicast traffic, and exchanging TLSv1 encrypted application data with `17.172.239.31`.

```
(garba@kali)-[~/SBT-DF203-Lab9]
$ tshark -r "$DEC" -Y 'ip' -T fields \
  -e frame.number -e wlan.sa -e wlan.da -e ip.src -e ip.dst -e _ws.col.Protocol \
  | head -n 200 | tee reports/ip_mac_mapping_sample.tsv
7      0.0.0.0 255.255.255.255 DHCP
9      0.0.0.0 255.255.255.255 DHCP
11     192.168.0.1 255.255.255.255 DHCP
20     192.168.0.15 8.8.8.8 DNS
21     192.168.0.15 8.8.8.8 DNS
22     192.168.0.15 224.0.0.2 IGMPv2
23     192.168.0.15 224.0.0.251 IGMPv2
24     192.168.0.15 224.0.0.2 IGMPv2
25     192.168.0.15 224.0.0.251 IGMPv2
26     8.8.8.8 192.168.0.15 DNS
27     8.8.8.8 192.168.0.15 DNS
30     192.168.0.15 17.172.239.31 TCP
31     192.168.0.15 17.172.239.127 TCP
33     17.172.239.31 192.168.0.15 TCP
35     192.168.0.15 17.172.239.31 TCP
36     192.168.0.15 17.172.239.31 TLSv1
37     17.172.239.127 192.168.0.15 TCP
38     192.168.0.15 17.172.239.127 TCP
42     17.172.239.31 192.168.0.15 TCP
43     17.172.239.31 192.168.0.15 TLSv1
44     17.172.239.31 192.168.0.15 TLSv1
45     17.172.239.31 192.168.0.15 TLSv1
47     0.0.0.0 255.255.255.255 DHCP
48     0.0.0.0 255.255.255.255 DHCP
49     192.168.0.15 17.172.239.31 TCP
50     192.168.0.15 17.172.239.31 TCP
51     192.168.0.1 192.168.0.15 DHCP
52     192.168.0.15 17.172.239.31 TLSv1
55     17.172.239.31 192.168.0.15 TCP
56     192.168.0.15 17.172.239.31 TLSv1
57     17.172.239.31 192.168.0.15 TCP
58     17.172.239.31 192.168.0.15 TLSv1
59     192.168.0.15 17.172.239.31 TCP
60     192.168.0.15 17.172.239.31 TLSv1
61     17.172.239.31 192.168.0.15 TLSv1
65     192.168.0.15 17.172.239.31 TCP
66     192.168.0.15 17.172.239.31 TLSv1
```

Protocol Execution & Decrypted Stream Logs

Terminal Command Executed:

```
tshark -r "$DEC" --export-objects http,exported/http_objects 2>&1 | tee reports/http_export_log.txt
```

Technical Evidence Analysis:

TShark stream analysis documents lower-level network initialization: ARP resolution requests

(`Who has 192.168.0.15?`), ICMPv6 router solicitations, DNS responses mapping `www.apple.com`, and TCP SYN handshakes on port 443.

```
(garba@kali) [~/SBT-DF203-Lab9]
$ mkdir -p exported/http_objects exported/foremost

(garba@kali) [~/SBT-DF203-Lab9]
$ tshark -r *SOCC --export-objects http_exported/http_objects 2>&1 | tee reports/http_export_log.txt
1 0.000000 EFMNetworks_55:97:d4 → Broadcast ARP 42 Who has 192.168.0.1? Tell 192.168.0.1
2 0.943674 Apple_68:96:7c → NetisTechnol_48:e5:b4 ARP 42 Who has 192.168.1.1? Tell 192.168.1.41
3 0.944186 Apple_68:96:7c → NetisTechnol_48:a8:b5 ARP 42 Who has 192.168.0.1? Tell 192.168.0.15
4 0.944186 Apple_68:96:7c → 08:10:77:92:7c:2f ARP 42 Who has 192.168.1.1? Tell 192.168.1.12
5 0.944186 Apple_68:96:7c → EFMNetworks_55:97:d4 ARP 42 Who has 192.168.0.1? Tell 192.168.0.15
6 0.944186 Apple_68:96:7c → TendaTechnol_58:c4:ba ARP 42 Who has 192.168.10.1? Tell 192.168.10.113
7 0.944185 0.0.0.0 → 255.255.255.255 DHCP 342 DHCP Request - Transaction ID 0x21e6be1c
8 0.944704 EFMNetworks_55:97:d4 → Apple_68:96:7c ARP 42 192.168.0.1 is at 00:26:66:55:97:d4
9 0.947778 0.0.0.0 → 255.255.255.255 DHCP 342 DHCP Request - Transaction ID 0x21e6be1c
10 0.949306 fe80::1c0c:f6e8:fedb:8f8e → ff02::2 ICMPv6 70 Router Solicitation from f0:f6:1c:68:96:7c
11 0.954435 192.168.0.1 → 255.255.255.255 DHCP 590 DHCP NAK - Transaction ID 0x21e6be1c
12 0.954937 Apple_68:96:7c → Broadcast ARP 42 ARP Announcement for 192.168.0.15
13 0.955970 fe80::1c0c:f6e8:fedb:8f8e → ff02::2 ICMPv6 70 Router Solicitation from f0:f6:1c:68:96:7c
14 0.956994 Apple_68:96:7c → Broadcast ARP 42 ARP Announcement for 192.168.0.15
15 0.959034 Apple_68:96:7c → Broadcast ARP 42 Who has 169.254.255.255? Tell 192.168.0.15
16 0.960066 Apple_68:96:7c → Broadcast ARP 42 Who has 169.254.255.255? Tell 192.168.0.15
17 1.026106 Apple_68:96:7c → Broadcast ARP 42 Who has 192.168.0.1? Tell 192.168.0.15
18 1.029698 Apple_68:96:7c → Broadcast ARP 42 Who has 192.168.0.1? Tell 192.168.0.15
19 1.029696 EFMNetworks_55:97:d4 → Apple_68:96:7c ARP 42 192.168.0.1 is at 00:26:66:55:97:d4
20 1.030202 192.168.0.15 → 8.8.8.8 DNS 73 Standard query 0x9583 A www.apple.com
21 1.186368 192.168.0.15 → 8.8.8.8 DNS 85 Standard query 0xbe40 A 48-courier.push.apple.com
22 1.187392 192.168.0.15 → 224.0.0.2 IGMPv2 46 Leave Group 224.0.0.251
23 1.187392 192.168.0.15 → 224.0.0.251 IGMPv2 46 Membership Report group 224.0.0.251
24 1.187904 192.168.0.15 → 224.0.0.2 IGMPv2 46 Leave Group 224.0.0.251
25 1.189439 192.168.0.15 → 224.0.0.251 IGMPv2 46 Membership Report group 224.0.0.251
26 1.350208 8.8.8.8 → 192.168.0.15 DNS 203 Standard query response 0x9583 A www.apple.com CNAME www.isg-apple.com.akadns.net CNAME www.apple.com.e
dgetkey.net CNAME e3191.dscc.akamaiedge.net A 104.74.154.228
27 1.350718 8.8.8.8 → 192.168.0.15 DNS 299 Standard query response 0xbe40 A 48-courier.push.apple.com CNAME 48.courier-push-apple.com.akadns.net C
NAME us-courier.push-apple.com.akadns.net A 17.172.239.31 A 17.172.239.127 A 17.172.232.188 A 17.172.239.83 A 17.172.232.166 A 17.172.239.25 A 17.172.232.17
0 A 17.172.232.185
28 1.350720 Apple_68:96:7c → Broadcast ARP 42 Who has 169.254.255.255? Tell 192.168.0.15
```

COMPREHENSIVE RECONSTRUCTION OF NETWORK TOPOLOGY

Correlating the decrypted stream across screenshots 468 through 471 establishes the following operational topology:

1. Internal Target Workstation:

- **IPv4 Assigned Address:** 192.168.0.15
- **Hardware MAC Address:** 68:96:7B:C8:2A:7A (Apple, Inc.)
- **Network Profile:** Primary workstation generating 98.7% of total network volume.

2. Wireless Infrastructure / Gateway Router:

- **Access Point BSSID:** 00:26:66:55:97:D6 (EFM Networks)
- **Default Gateway IP:** 192.168.0.1

3. External Server Destination Nodes:

- 198.90.20.111:80 (HTTP) Primary remote web host receiving bulk HTTP GET requests.
- 199.27.79.193:80 (HTTP) Secondary web application and media server.
- 17.172.239.31:443 (TLS/SSL) Remote Apple push notification and cloud sync service endpoint.

DIGITAL ARTIFACT EXTRACTION & FILE CARVING

Two distinct extraction methodologies were deployed to recover transmitted digital artifacts: protocol-aware object extraction via tshark and header-footer file carving via foremost.

Foremost Execution & File Type Identification

Terminal Command Executed:

```
foremost -i "$DEC" -o exported/foremost | tee reports/foremost_log.txt
```

```
find exported -type f -exec file {} \; | tee reports/exported_file_types.txt
```

`foremost` successfully carved binary structures out of unencrypted payload streams. Execution

of `file` classifies recovered artifacts into JSON documents, JavaScript source files (`widgets.js`), PNG images (`Figure-2.png`), and CSS stylesheets.

```
(garba@kali) [~/SBT-DF203-Lab9]
$ foremost -i "$DEC" -o exported/foremost | tee reports/foremost_log.txt
Processing: working/file_working-dec
[*]

(garba@kali) [~/SBT-DF203-Lab9]
$ find exported -type f -exec file {} \; | tee reports/exported_file_types.txt
exported/http_objects/hit(3).json: JSON text data
exported/http_objects/collapsible-widgets.js: JavaScript source, ASCII text, with very long lines (7085), with CRLF line terminators
exported/http_objects/sha1.js: ASCII text
exported/http_objects/hls%3fversion=4.12: XML 1.0 document, ASCII text
exported/http_objects/imageview.gif%3fa=ogli5PAfr=http%3A%2F%2Fm.reddit.com%2F: ASCII text, with no line terminators
exported/http_objects/Figure-2.png: PNG image data, 537 x 274, 8-bit/color RGBA, non-interlaced
exported/http_objects/YDAoLskQ5MOAgvHUQCcLQa6gm6bS00u2Qn-iPLo1Go.woff: Web Open Font Format, TrueType, length 22824, version 1.1
exported/http_objects/trackalyze.js: ASCII text, with CRLF line terminators
exported/http_objects/jquery-1.10.2.min.js: JavaScript source, ASCII text, with very long lines (32072)
exported/http_objects/adc-i-00aecd.png: PNG image data, 12 x 12, 8-bit/color RGBA, non-interlaced
exported/http_objects/v2(1): JSON text data
exported/http_objects/lumbar.gif%3fa=%7B%22UID%22%3A%222ae1543dcd8ab603f50e1849b3db2c79e9ce5872c0878dfc88d838afa4c1a3d%22%2C%22accountId%22%3Anull%2C%22sessionId%22%3A%224d0d205ea8f1f11bd5b098028b761dc%22%2C%22platform%22%3A%22web%22%2C%22source%22%3A%22(1).com%2F8iVv7t6%22%7D: ASCII text, with no line terminators
exported/http_objects/jquery-ui.css: ASCII text, with very long lines (551)
exported/http_objects/lumbar.gif%3fa=%7B%22UID%22%3A%222ae1543dcd8ab603f50e1849b3db2c79e9ce5872c0878dfc88d838afa4c1a3d%22%2C%22accountId%22%3Anull%2C%22sessionId%22%3A%224d0d205ea8f1f11bd5b098028b761dc%22%2C%22platform%22%3A%22web%22%2C%22source%22%3A%22(3).com%2F8iVv7t6%22%7D: ASCII text, with no line terminators
```

Carved Evidence Directory & Hash Verification

Terminal Command Executed:

```
find exported -type f -exec sha256sum {} \; | tee reports/exported_file_hashes.txt
```

Directory listing logs recovered image files (`00001831.png`, `00002424.png`) and HTML documents

(`00024427.htm`). SHA-256 calculation verifies that all carved artifacts are cryptographically recorded for courtroom presentation.

[illegible]

REQUIRED FINDINGS WORKSHEET

Assessment Question / Finding Category **Verified Forensic Observation / Technical Value**

Capture Format & Duration	TCPdump pcap (little-endian), 272.97 seconds duration (45,169 total packets)
Wireless Access Point BSSID	00:26:66:55:97:D6 (Vendor: EFM Networks)
Primary Client MAC Address	68:96:7B:C8:2A:7A (Vendor: Apple, Inc.)
Protected Frame Count	15,477 IEEE 802.11 encrypted data frames
Repeated IV Evidence	High-density IV reuse verified (e.g., IV 0x4208cb and 0x28d3eb repeated 7 times each)
Recovered WEP40 Secret Key	A4:3D:F6:F3:74 (Airdecap input parameter: A43DF6F374)
Decrypted Capture File Path	working/file_working-dec
Decrypted Capture SHA-256	167c91994c269777f9048227dcb89882caf3cf763977f2059604f0a6a340b804
Host IP & MAC Mapping	Client IP: 192.168.0.15 Client MAC: 68:96:7B:C8:2A:7A

Server IP & AP Gateway	Remote Server IP: 198.90.20.111 Default Gateway IP: 192.168.0.1
Key Network Protocols Identified	IEEE 802.11, ARP, IPv4, IPv6, DNS, IGMPv2, TCP, HTTP, TLSv1
Extracted Files / Artifacts	PNG graphics (Figure-2.png), JS (widgets.js), HTML (00024427.htm), JSON payloads
Security Conclusion	WEP is cryptographically broken due to short IV length causing rapid key-recovery exposure.

CRYPTOGRAPHIC SECURITY ANALYSIS & MITIGATION CONTROL STANDARDS

Structural Cryptographic Flaws in WEP

- Inadequate Initialization Vector Space:** The 24-bit cleartext IV field guarantees that under standard packet generation rates, IV collisions occur rapidly.
- Keystream Reuse & FMS Exposure:** Reusing an IV with a static key creates identical RC4 keystreams. Collecting thousands of frames allows attackers to bypass key length entirely using statistical correlation attacks.
- Unauthenticated CRC-32 Integrity Checks:** WEP relies on CRC-32 (ICV) for frame integrity, which is non-cryptographic and linear. This design allows malicious bit-flipping modifications without invalidating the checksum.

REMEDIATION & INSTITUTIONAL RECOMMENDATIONS

- Decommission Legacy WEP Deployments:** WEP cannot be secured by expanding passphrase length or shortening rotation windows. All legacy installations must be retired immediately.
- Transition to Modern Encryption Protocols:** Deploy **WPA2-Enterprise (AES/CCMP)** or **WPA3-Enterprise (GCMP-256 / SAE)**. Modern standards enforce temporal key integrity protocols (TKIP/AES) that generate unique per-packet keys, rendering IV collection attacks ineffective.

- **Implement Protected Management Frames (PMF):** Mandate IEEE 802.11w PMF to protect management frames against rogue deauthentication attacks and frame injection.

ACADEMIC INTEGRITY & AUTHORIZATION STATEMENT

I confirm that I completed this lab in an authorized virtual environment using the supplied historical CTF dataset. I did not target any third-party network, and I have accurately documented my own commands, hashes, screenshot evidence, and observations.

Student Name: Ahmad Zubairu Garba

Date: September 26, 2026